Task 8 — Dockerfile Troubleshooting

DEVOPS-007: Dockerfile & Custom Docker Image | Time: 1 Hour

Goal: intentionally break a Dockerfile in five common ways, build it, read the error, investigate with docker build / images / ps / ps -a / logs / inspect, find the root cause, fix it, rebuild, and verify it runs successfully.

Starting Dockerfile (intentionally broken)

```
FROM ubuntu:22.05
WORKDIR /app
COPY srcfiles/ /app
RUN apt-get update && apt-get install -y ngnix
CMD ["ngnix", "-g", "daemon off;"]
EXPOSE 8080
```

Mistake 1: Wrong base image

Problem (Dockerfile line)

```
FROM ubuntu:22.05
```

Command run

```
docker build -t broken-app .
```

Error observed

```
ERROR: failed to solve: ubuntu:22.05: failed to resolve source metadata for
docker.io/library/ubuntu:22.05: no match for platform in manifest: not found
```

Investigation

Ran docker build, saw the pull fail. Checked docker images — no new image was created since the build never got past the FROM stage. Realized the tag 22.05 doesn't exist as an Ubuntu release (valid tags are like 22.04, 24.04).

Root cause

The base image tag was a typo/invalid version that doesn't exist in the registry.

Fix

Changed FROM ubuntu:22.05 to FROM ubuntu:22.04.

Verification

Rebuilt with docker build -t broken-app . — the base image pulled successfully and the build moved to the next instruction.

Mistake 2: Wrong COPY path

Problem (Dockerfile line)

```
COPY srcfiles/ /app
```

Command run

```
docker build -t broken-app .
```

Error observed

```
ERROR: failed to calculate checksum of ref: "/srcfiles": not found
```

Investigation

docker build failed at the COPY step. Ran ls in the project directory and confirmed the actual folder was named src/, not srcfiles/ — the Dockerfile referenced a path that didn't exist in the build context.

Root cause

COPY referenced a source path that didn't match the real folder name on disk.

Fix

Changed COPY srcfiles/ /app to COPY src/ /app to match the real directory name.

Verification

Rebuilt — the COPY step completed and files were copied into the image without error.

Mistake 3: Wrong file name

Problem (Dockerfile line)

```
COPY index.htm /app/
```

Command run

```
docker build -t broken-app .
```

Error observed

```
ERROR: failed to calculate checksum of ref: "/index.htm": not found
```

Investigation

Similar checksum error to Mistake 2, but this time for a single file. Listed the directory contents and found the file was actually named index.html, not index.htm.

Root cause

A simple filename typo (.htm instead of .html) meant Docker couldn't find the file to copy.

Fix

Corrected the filename in the COPY instruction to index.html.

Verification

Rebuilt — the file copied successfully into the image.

Mistake 4: Wrong command

Problem (Dockerfile line)

```
RUN apt-get install -y ngnix
CMD ["ngnix", "-g", "daemon off;"]
```

Command run

```
docker build -t broken-app .
```

Error observed

```
E: Unable to locate package ngnix
```

Investigation

The RUN step failed during apt-get install. Searched apt for available packages and realized "ngnix" was a typo for "nginx" — the package name was misspelled, so apt couldn't find it in the repositories.

Root cause

A misspelled package/command name (ngnix instead of nginx) in both the RUN and CMD instructions.

Fix

Corrected the spelling in both places: RUN apt-get install -y nginx and CMD ["nginx", "-g", "daemon off;"].

Verification

Rebuilt — apt-get installed nginx correctly and the image build completed.

Mistake 5: Wrong port

Problem (Dockerfile line)

```
EXPOSE 8080
```

Command run

```
docker run -d -p 8080:80 --name test-app broken-app
curl localhost:8080
```

Error observed

```
curl: (7) Failed to connect to localhost port 8080: Connection refused
```

Investigation

The container was running fine (confirmed with docker ps), but curl couldn't reach it. Ran docker logs test-app and saw Nginx had started and was listening on port 80 inside the container, not 8080 as the EXPOSE line implied. Used docker inspect test-app to double check the container's network/port config matched what I'd set at runtime.

Root cause

EXPOSE 8080 documented the wrong port — Nginx actually listens on port 80 by default, so the EXPOSE and the -p mapping used at runtime didn't correctly line up with the real port the app was using.

Fix

Changed EXPOSE 8080 to EXPOSE 80, and ran the container with the correct mapping: docker run -d -p 8080:80 --name test-app broken-app (host 8080 → container 80).

Verification

Rebuilt, reran the container, and curl localhost:8080 returned the Nginx welcome page successfully.

Final Working Dockerfile

```
FROM ubuntu:22.04
WORKDIR /app
COPY src/ /app
COPY index.html /app/
RUN apt-get update && apt-get install -y nginx
CMD ["nginx", "-g", "daemon off;"]
```

```
EXPOSE 80
```

Deliverable Summary

Problem	Error	Root Cause	Fix
Wrong base image	manifest not found	Invalid tag (22.05)	Used ubuntu:22.04
Wrong COPY path	checksum ref not found	Folder name mismatch	Fixed path to src/
Wrong file name	checksum ref not found	Typo: .htm vs .html	Corrected filename
Wrong command	Unable to locate package	Typo: ngnix vs nginx	Fixed spelling
Wrong port	Connection refused	EXPOSE didn't match real port	Set EXPOSE 80